ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/370854448

The Effects of Worked-Out Example and Metacognitive Scaffolding on Problem-Solving Programming

Article *in* Journal of Educational Computing Research · May 2023

DOI: 10.1177/07356331231174454

CITATIONS
0

READS
129

4 authors, including:

Jaewon Jung
Korean Educational Development Institute
5 PUBLICATIONS **21** CITATIONS

SEE PROFILE

Joerg Zumbach
University of Salzburg
117 PUBLICATIONS **1,560** CITATIONS

SEE PROFILE

All content following this page was uploaded by Joerg Zumbach on 23 May 2023.

The user has requested enhancement of the downloaded file.

Check for updates

Original Research Article

Journal of Educational Computing Research
2023, Vol. 0(0) 1–20
© The Author(s) 2023
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/07356331231174454
journals.sagepub.com/home/jec
Sage

The Effects of Worked-Out Example and Metacognitive Scaffolding on Problem-Solving Programming

Yoonhee Shin[1], Jaewon Jung[2], Joerg Zumbach[3], and Eunseon Yi[4]

Abstract

This study explores the effects of worked-out examples and metacognitive scaffolding on novice learners' knowledge performance, cognitive loads, and self-regulation skills in problem-solving programming. 126 undergraduate students in a computer programming fundamentals course were randomly assigned to one of four groups: 1) task performance with a traditional WOE (TW), 2) task performance with a faded WOE (FW), 3) task performance with traditional WOE and metacognitive scaffolding (TWM), and 4) task performance with a faded WOE and metacognitive scaffolding (FWM). Over the course of 3 weeks, participants in these four groups were asked to solve programming problems using Python with WOE and metacognitive scaffolding. The results demonstrate that the provision of metacognitive scaffolding with faded WOE (FWM) is the most effective for problem-solving programming and self-regulation skills. In addition, an interaction effect exists between the two treatments for the germane load in FWM. Therefore, results in this study provide empirical

[1]College of Education, Hanyang University, Seoul, Republic of Korea
[2]Korean Educational Development Institute (KEDI), Jincheon-gun, Republic of Korea
[3]E-Learning in Science Education, School of Education, University of Salzburg, Salzburg, Austria
[4]Department of Creative Convergence Education and College of Computer Science, Hanyang University ERICA, Ansan, Republic of Korea

Corresponding Author:
Jaewon Jung, Korean Educational Development Institute (KEDI), Gyohak-ro, Deoksan-eup, Jincheon-gun, Chungcheongbuk-do, Jincheon-gun 27873, Republic of Korea.
Email: jungj5@kedi.re.kr

insights into ways to effectively apply WOE and metacognitive scaffolding to problem-solving processes for programming-based complex problem-solving, especially for novice learners.

Keywords
problem-solving programming, self-regulation, cognitive load, metacognitive scaffolding, faded woe

Introduction

Problem-solving skills that leverage computational thinking are a core competency in the 21st century (Haseski et al., 2018), and, in higher education, attempts have emerged to apply programming to problem-solving. Learning strategies that combine programming with complex problems are effective not only in identifying and analyzing complex problems, but also in applying computing principles to new problem situations (Agbo et al., 2019). However, understanding computational principles and utilizing them to solve problems requires self-regulation consisting of cognitive and metacognitive skills. Novices often give up on programming when they fail to effectively apply these learning skills during problem-solving processes (Magana et al., 2019).

Previous studies on programming education have reported that the learning skills of experts, as compared to those of novices, include knowing how to plan, monitor, and reflect on their problem-solving process (Chao, 2016; Loksa et al., 2016). These are not only important in troubleshooting programming, but they are also necessary from problem identification to the implementation process. However, novices struggle to develop these skills themselves and to use them (Loksa et al., 2020).

Recently, growing interest has focused on learning programming for problem-solving, with a special focus on the effects of metacognitive scaffolding on programming (Shin & Song, 2022). Some studies investigated the role of metacognitive skills in problem-solving (Loksa & Ko, 2016; Pedrosa et al., 2016; Xie et al., 2019). The findings showed that novices put a lot of effort into identifying and interpreting problems in the early stage of problem-solving according to the planning strategy, which was also associated with high knowledge acquisition.

However, these also demonstrated that metacognitive support was effective when learners had sufficient prior knowledge related to the problem. These further indicate the necessity of metacognitive scaffolding for novices to analyze complex problems and flexibly apply prior knowledge, as well as the importance of scaffolding that considers cognitive aspects at the beginning of learning.

In programming learning, cognitive support such as providing worked-out example (WOE) has been used to support prior knowledge acquisition for novice learners (Garner, 2002) In addition, research has shown that WOE can be more effective when

using self-explanations (Crippen & Earl, 2007). However, WOE does not always aid in problem-solving or knowledge transfer in steps beyond the acquisition of knowledge (Sweller et al., 1998). Even if a WOE is provided to support the cognitive aspect, an instructional design that carefully considers learners' cognitive loads is required to link with self-regulation skills (Moreno, 2006). Among WOE instructional strategies, faded WOE is effective in helping learners with the self-directed acquisition of knowledge and the expansion of critical thinking through trial and error (Renkl, 2002). In the problem-solving process, it has been suggested that WOE should be provided in a faded form to effectively transfer knowledge to prevent unnecessary cognitive load (Garner, 2001; Salleh et al., 2018). However, not enough research exists to determine how effective a faded WOE is in programming-based troubleshooting.

To support novices' programming learning using problem-solving, it is necessary to provide scaffolding that systematically facilitates self-regulation, including both cognitive aspects and metacognitive aspects. However, compared to the expansion of programming problem-solving learning, studies on cognitive and metacognitive strategies to promote knowledge performance including acquisition and problem-solving skills for novice learners are currently insufficient. In addition, there is a lack of studies that closely explore the effects of cognitive and metacognitive scaffolding on self-regulation in programming learning. In this context, this study investigates the effect of the WOE and metacognitive scaffoldings, considering the characteristics of the problem-solving phase, on the promotion of knowledge acquisition, problem-solving, cognitive load, and self-regulation among novices.

The study aims to answer the following research question: Do the two types of scaffolding (Faded WOE, Metacognitive Scaffolding) provided in the programming problem-solving process evidence a significant difference in the level of knowledge performance, cognitive load, and self-regulation skills?

Literature Review

Problem-solving Using Programming

As the number of innovative problem-solving practices using computer programming increases, there have been many attempts to effectively promote skills for programming problem-solving skills, a complex activity that requires learners to perform various cognitive processes (Loksa et al., 2016; Prather et al., 2020). It can be largely divided into two phases: problem-analysis and problem-solving (Loksa & Ko, 2016). In the former, learners interpret the problem and clarify the task required to solve the problem. Then learners find a problem similar to the proposed problem and infer a solution. In this phase, learners connect the problems experienced in the past to the new problem situation. Thus, it is necessary to be able to collect and analyze various resources related to the proposed problem. On the other hand, the problem-solving phase involves evaluating the results of repeated implementations (Chao, 2016).

According to a study that analyzed the problem-solving patterns of experts and novices in the programming problem-solving process, experts showed a problem-solving process centered on clarifying knowledge and planning to solve the problem. In the problem-analysis phase, the experts clearly understood the problem and established a solution plan by dividing acquired knowledge from unacquired knowledge (Lister et al., 2006; Xie et al., 2019). Based on this, experts accurately applied the knowledge, discovered new knowledge, and expanded it in the problem-solving phase. On the other hand, novices implemented the solution without any strategies for problem-solving, making simple and repetitive mistakes in the process (Magana et al., 2019). Because the cognitive process is quite abstract and complex due to the nature of programming problem-solving, novices often give up on problem-solving learning.

Magana et al. (2019) reported that a systematic guide to help learners gradually master strategies is required for any complex cognitive process. Given the interrelationship between the cognitive and metacognitive aspects, it is necessary to provide scaffolding that helps novices not only to understand the knowledge required by the problem in the problem-analysis phase (Stachel et al., 2013; Yousoof et al., 2007), but also to plan, monitor, and reflect through the whole problem-solving process. Given that experts use cognitive and metacognitive skills appropriately in complex problem-solving through a high level of self-regulation skills (Loksa et al., 2016), the connection between cognitive and metacognitive processes is very important. Thus, it is necessary to consider how and when to support cognitive and metacognitive scaffolding to function effectively in programming problem-solving.

Metacognitive Scaffolding for Problem-solving

In programming learning, metacognitive skill is important in that it affects the acquisition and expansion of new knowledge by effectively applying the learners' prior knowledge to problem-solving (National Academies of Sciences, 2018). In general, highly self-regulated experts have not only cognitive skills, but also high levels of metacognitive skills (Loksa et al., 2016; Zimmerman & Schunk, 2011). In the troubleshooting process, metacognitive scaffolding such as planning, monitoring, and reflection should be properly supported to promote metacognitive skills (Molenaar et al., 2011; Prather et al., 2020). The problem-solving process can be generally divided into a problem-analysis phase and a problem-solving phase (Loksa & Ko, 2016). In the problem-analysis phase, the learner analyzes what knowledge is necessary to solve the problem, what concepts he or she knows, and what new concepts should be known in order to find a solution. In the problem-solving phase, the learner embodies the solution, reflects on whether the solution is correct, and continuously corrects the solution.

Previous studies on metacognitive strategies proved that metacognitive skills through metacognitive prompts such as planning, monitoring, and reflection were effective for problem-solving. However, most studies focused on providing metacognitive scaffolding only in the "problem-solving" phase, which is the latter part of the

problem-solving process. Pedrosa et al. (2016) supported the organization and planning activities in the early part of the problem-analysis process and the application of theoretical knowledge in the problem-solving process. The findings indicated that metacognitive scaffolding was effective in designing how to approach a problem, selecting elements such as concepts or principles to be used for the solution, and coordinating how to combine two or more elements. However, it was suggested that both metacognitive scaffolding and cognitive scaffolding promote cognitive skills such as use, understanding, and information should be provided for effective problem-solving.

While studies on providing scaffolding suitable for the characteristics of each phase by dividing the problem-solving process into phases are becoming more common, previous studies tended to focus on providing all the metacognitive scaffoldings at the beginning of the problem-solving process, rather than on providing classified and sequential scaffoldings suitable for each phase (Loksa & Ko, 2016). In addition, most studies centered on implementing either cognitive scaffolding or metacognitive scaffolding and measuring problem-solving outcomes. Therefore, for successful problem-solving, research is needed to explore scaffolding that supports both the cognitive and metacognitive aspects, even as it offers systematic support step by step, including the problem analysis phase of the problem-solving process.

Worked-out Example to Promote Cognitive Skill

According to the cognitive load theory (CLT), the learners' cognitive structure determines the amount of information that can be processed at one time in working memory (Sweller et al., 1998). Cognitive load is generally divided into intrinsic cognitive load, extraneous cognitive load, and germane cognitive load. Cognitive overload occurs when the total amount of cognitive load exceeds the amount of cognitive load that a learner can handle in working memory (Paas et al., 2004). While intrinsic and extraneous cognitive load are unnecessary cognitive load, germane cognitive load occurs in the process of automation and schema construction. The last is generative and directly related to learning (Jung et al., 2016). Thus, previous studies on cognitive load have proposed instructional strategies that minimize intrinsic and extrinsic cognitive load and maximize the germane cognitive load that directly contributes to learning (Paas et al., 2003).

Learners experience cognitive difficulties when the content of a learning task is complex (Jung et al., 2019b). In programming learning, WOE can be used as a way to reduce the complexity of the content of learning tasks. Solving problems requires closing the gap between what learners actually know and what they need to know (Paas & Van Merriënboer, 1994). Specifically, WOE for programming learning provides the important concepts and main solution principles required by the problem in connection with each other, which helps the learner to infer solutions to new problems (Vieira et al., 2015). In particular, WOEs that fade some of the content of the expert's problem-solving process guides learners to accurately acquire important concepts and principles

required at each phase for problem-solving (Chi et al., 1989; Jung et al., 2019a; Renkl, 2002). Studies on faded WOE found that WOEs with some information faded had a positive effect on learning transfer (Renkl et al., 2000, 2004).

However, despite the effects of such faded WOE, some studies suggest that faded WOE induces cognitive difficulties leading to ineffective learning when considering the complexity of the programming problem-solving process (Paas, 1992). This inconsistency in study results for different types of WOE is due to the diversity of learner levels and characteristics. Thus, it is necessary to explore what kind of WOE type (i.e., a completed WOE or a faded WOE) is suitable, considering the learner's learning level (i.e., novice or more experienced) (Abdul-Rahman & Du Boulay, 2014; Ge et al., 2016; Puntambekar & Hubscher, 2005). However, in programming problem-solving learning, there are not enough studies that investigate the WOE type to promote learners' cognition that considers learning level. Therefore, in this study, we try to explore WOE types that support problem-solving for novices by providing cognitive scaffolding that deletes some important parts of concepts and principles necessary for problem-solving.

Methods

Participants

Participants in this study were 126 first-year university students majoring in humanities and social sciences at a university in Korea. All participants were students enrolled in the 2-credit course "Fundamental Python Programming," and none had experience taking courses related to Python programming. During the class orientation, all participants were provided with an explanation of the study's purpose, procedures, benefits, data usage, and their rights as research participants, and only those who consented were included in this study. The participant age range was from 20 to 22 years old, with a mean age of 20.5 years. Each weekly class included an explanatory lecture, and each participant used a web-based programming tool to perform programming assignments. Participants were randomly assigned to four groups after pretesting of prior knowledge. Group TW performed tasks with a traditional WOE (n = 30). Group FW performed tasks with a faded WOE (n = 30). Group TWM performed tasks with a WOE and metacognitive scaffolding (n = 33). Group FWM performed tasks with a faded WOE and metacognitive scaffolding (n = 33).

Experiment Design

This experiment was conducted after participants had spent 3 weeks learning basic programming concepts (e.g., variables, loops, conditionals, functions, and lists). The instructor guided learners to practice through explanations using algorithms and five simple code examples for each concept. For example, in the phase of learning conditional statements, the problem of "open the name list if the input data is correct and close if the input data is incorrect" was used. All four groups studied the same learning

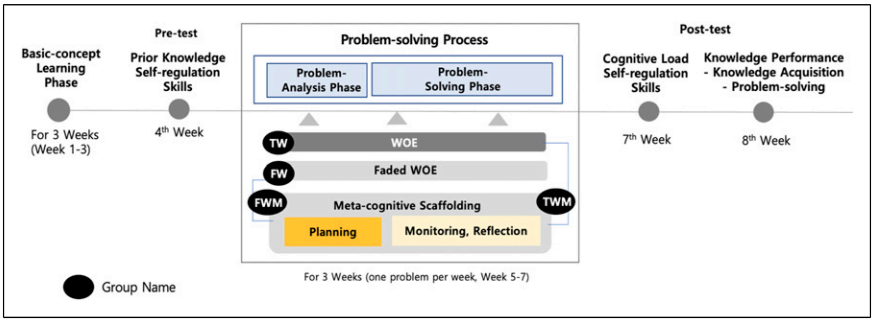

Figure 1. Experiment design.

materials. The experiment itself was conducted in two-hour segments over the course of eight total weeks. Before the experiment, we conducted two types of prior tests, including prior knowledge and self-regulation skills. The items for the prior knowledge and knowledge acquisition test were referenced and partially revised from a Python tutorial (van Rossum & Python development team, 2018). The problem-solving tasks were jointly revised to fit the study context from a structured problem in the python tutorial by the authors of this research. The tasks were evaluated for their appropriateness in real-world settings by three experts with more than 5 years of experience in teaching computer programming and educational technology, and was subsequently modified based on their feedback. During the 3 weeks of the problem-solving process, students were presented with semi-open form problems which were related to real-world situation allowing the derivation of various solutions. The issues presented every week involved a movie reservation program, a lunch menu recommendation program, and a shopping mall order program. Each of these included a problem-solving context (e.g., the development of a sales management program to address a situation in which employees working at a movie theater suffer from poor organization). During the problem-solving process, we conducted a problem-solving process divided into problem-analysis and problem-solving phases (see Figure 1).

In the cases of the TWM and FWM groups, both of which were provided with metacognitive scaffolding, group members read questions and suggestions for 30 min during the problem analysis phase and then wrote planning ideas. The problem-solving phase took 1 hr and 30 min to write the code with reference to the metacognitive scaffolding, which included a monitoring and reflection guide. After the problem was solved, the notes created during the planning phase were collected, but the students were informed that these notes would not be incorporated into their grades. The problem-solving code for both groups was submitted as a file, with explanation comments that learners wrote during monitoring and reflection of the metacognitive phase. The TW and FW groups were not provided with metacognitive scaffolding to solve the problem for 2 hours without distinguishing the problem-solving phases, and they were allowed, for 2 hours, to freely approach problems without special restrictions

in the problem-solving phase. However, the FW was given a progressively faded form of the main content for 3 weeks. After submitting the programming task in the seventh week, students submitted a cognitive load and self-regulation skill questionnaire. In the 8th week, a programming knowledge performance test, including knowledge acquisition and problem-solving, was presented over 2 hours.

Two Types of Scaffolding

Metacognitive Scaffolding. To support novices, we divided the programming problem-solving process into two phases: problem-analysis and problem-solving. We then applied several metacognitive scaffoldings each by itself: metacognitive scaffolding for planning in the problem-analysis phase, and metacognitive scaffolding for monitoring and reflection in the problem-solving phase. The scaffolding included planning, monitoring, and reflection questions. The planning guide was designed to help learners better understand problem descriptions and the concepts required to solve the relevant problems. The monitoring and reflection guides included directions on how to review results and explore conditions for error correction. Specifically, the metacognitive questions consisted of references from Loksa (2016), as follows.

- Planning: (1) What is required to solve the problem? (2) What variables and key phrases are needed to solve the problem? (3) Is the new information presented in the question different from the information understood?
- Monitoring: (1) Has the detailed goal setting for problem-solving been completed? (2) Has your code been sufficiently tested? (3) What do you think is the cause of the failure?
- Reflection: (1) Is there a course I missed? (2) What more do I need to learn in the future?

Faded Worked-out Example. Each WOE included a "problem" section that contained problem statements and objectives, along with a "solution" section. Problems included a problem-solving context, task, input data, and output data. Resolution steps included a basic program structure and expected program outcome. Using the fading method, in the first week, we provided a WOE that included code and comments. In the second week, we faded the line of steps that included the variable declaration in the introductory part, the basic syntax, and operators (e.g., input, keyword, loop). In the third week, we faded all the code, including comments, leaving only the "print" code. The group provided with WOE received a guide containing all the code, including print code and comments.

Measurement

Prior Knowledge. We designed the pre-test to assess the participants' prior knowledge. It consisted of 10 multiple-choice questions about basic Python concepts and syntax

Table 1. The Group Means of Prior Knowledge.

Groups	*N*	Mean	SD
TW (traditional WOE)	30	6.07	2.08
FW (faded WOE)	30	5.80	2.16
TWM (traditional WOE and metacognitive scaffolding)	33	6.70	2.28
FWM (faded WOE and metacognitive scaffolding)	33	6.39	2.19

(e.g., "Choose the one that is appropriate as a principle when declaring a variable. Choose the one that is appropriate for the description of the conditional statement"). One point was provided for a correct answer to each problem, with a maximum score of 10 points. The result shows that there was no statistically significant difference in the types of WOE ($F = .54$, $p = .47$, $\eta^2 = .00$) and metacognitive scaffolding ($F = 2.47$, $p = .12$, $\eta^2 = .02$), and interaction effects between two treatments ($F = .00$, $p = .96$, $\eta^2 = .00$) among four groups (see Table 1).

Knowledge Performance. Three weeks after the problem-solving study was completed, a programming knowledge performance test was conducted for a total of 2 hours. Knowledge performance was assessed using a knowledge acquisition and problem-solving test that utilized python programming.

Knowledge Acquisition. The acquisition test consisted of 2-point multiple-choice questions on the main concepts learned and 3-point filling in the blanks with a closed short answer. Ten multiple-choice questions addressed programming grammar (e.g., what was wrong with the description of the range () function). Five of the questions focused on presenting programming-related problems and filling in less-complete programs (e.g., fill in the blanks in programming, so that the following output appears using the 'list' syntax for slicing).

Problem-Solving. Problem-solving means that programming-related knowledge does not appear on the surface as it is and that it must be possible to select the learned knowledge contained in any problem or situation and then apply the learned knowledge to the solution. It is a complex form that can be appropriately leveraged to solve the problem. In other words, the situation suggested in the problem-solving task is contextual, complex, and abstract. The problem-solving task can be suggested as a problem of giving weight and height information at an amusement park and devising a system that judges whether it is possible to ride according to the input values. We suggested a set of problem-solving tasks such as a program to create a unit calculator to calculate the average and units, a restaurant inventory management program, and an amusement park program. The problem was presented along with the problem situation and the output result. As for the program-solving test, the programmed result completed at the basic level, such as the 'print' statement, was presented, and the method of filling

in five or more blanks in the program or finding and correcting the wrong place was a method. If all blanks matched the correct answer, a score was given. A total of 25 points were given for a total of five questions, five points per question.

Cognitive Load. Participants' cognitive load levels were measured using a 10-point Likert scale (Leppink et al., 2013), ranging from "Extremely Easy" (0) to "Extremely Difficult" (10). The cognitive load measurement consisted of 10 multiple-choice items (e.g., "The content of this learning task was very complex"); three items for intrinsic load, three for extraneous load, and four for germane load; Cronbach's Alpha = .83, .81, and .74, respectively).

Self-Regulation Skills. Self-regulation skills were measured using a subscale of the Motivated Strategies for Learning Questionnaire (MSLQ) related to motivation and self-regulated learning (Pintrich et al., 1991). MSLQ has already been widely used in computer science education research, and its validity and reliability have also been proven. For this study, the metacognitive self-regulation-related subscale was chosen to measure learners' self-regulation skills. The questionnaires consisted of 12 multiple-choice questions (e.g., "I ask myself questions to make sure I understand the material I have been studying in this class. When studying for this course, I try to determine which concepts I don't understand well. When I study for this class, I set goals for myself to direct my activities in each study period"). Questionnaires were measured in pre-test and post-test (Cronbach's Alpha = .82; .76, respectively).

Results

Knowledge Performance

To examine the effects of the provided scaffolding on knowledge performance, a two-way MANCOVA with was conducted, controlling for differences in prior knowledge, and self-regulation skills in the pre-test. Table 2 shows means and SD on knowledge acquisition and problem-solving scores.

Knowledge Acquisition. The multivariate results indicated a significant main effect of WOE scaffolding types ($F = 5.86$, $p < .05$; $\eta^2 = .05$), and a significant main effect for metacognitive scaffolding ($F = 16.23$, $p < .01$; $\eta^2 = .12$). This result indicates that TWM and FWM groups outperformed the comparison groups in knowledge acquisition; however, TWM which provided the traditional WOE showed a higher score than FWM which provided the faded WOE. Interaction between both variables was not significant ($F = .00$, $p = .98$; $\eta^2 = .00$).

Problem-Solving. The multivariate results indicated no significant main effect of WOE scaffolding types ($F = 1.87$, $p = .17$; $\eta^2 = .02$) and a significant main effect for metacognitive scaffolding ($F = 9.48$, $p < .01$; $\eta^2 = .07$). This result indicates that the

Table 2. Descriptive Statistics for Knowledge Performance and Self-Regulation Skills.

	Knowledge Performance		Cognitive Load			Self-Regulation Skills	
Group	Knowledge Acquisition	Problem-Solving	Intrinsic	Extraneous	Germane	Pre-test	Post-test
TW	47.67 (23.63)	10.67 (7.16)	6.26 (2.38	4.56 (1.92)	6.39 (1.86)	4.39 (0.74)	4.20 (0.90)
FW	38.97 (21.35)	9.82 (8.29)	6.89 (2.47)	4.37 (2.24)	6.12 (2.15)	4.34 (0.92)	4.28 (1.12)
TWM	61.52 (14.28)	13.03 (5.85)	4.80 (1.79)	3.58 (1.56)	6.30 (1.64)	4.29 (1.33)	4.61 (.72)
FWM	53.18 (14.02)	16.21 (5.73)	4.94 (2.47)	2.86 (1.52)	7.45 (1.22)	4.26 (0.87)	4.71 (0.72)

Table 3. The Two-Way Mancova Summary of Cognitive Load.

Variables	Aspect	SS	Df	MS	*F*	*p*	η^2
WOE scaffolding	Intrinsic load	2.56	1	2.56	.51	.48	.00
	Extraneous load	8.41	1	8.41	2.63	.11	.02
	Germane load	7.01	1	7.01	2.49	.12	.02
Meta scaffolding	Intrinsic load	76.06	1	76.06	15.19	.00	.11
	Extraneous load	39.90	1	39.90	12.47	.00	.10
	Germane load	12.25	1	12.25	4.35	.04	.04
Faded scaffolding* Meta scaffolding	Intrinsic load	1.46	1	1.46	0.29	.59	.00
	Extraneous load	1.85	1	1.85	0.58	.45	.00
	Germane load	15.08	1	15.08	5.35	.02	.04
Error	Intrinsic load	590.88	118	5.01			
	Extraneous load	377.54	118	3.20			
	Germane load	332.37	118	2.82			

FWM group most outperformed the comparison groups in problem-solving. Interaction between both variables was not significant ($F = 2.91$, $p = .09$; $\eta^2 = .02$).

Cognitive Load

Cognitive Load was assessed using a questionnaire at the end of the course instruction. To examine the effects of the provided scaffolding on cognitive load, a two-way MANCOVA with was conducted, controlling for differences in prior knowledge, and self-regulation skills in the pre-test. Table 2 shows means and SD on cognitive load which is divided into intrinsic, extraneous, and germane load.

Results reveal a significant main effect of metacognitive scaffolding on intrinsic cognitive load ($F = 15.19$, $p < .01$; $\eta^2 = .11$), extraneous load ($F = 12.47$, $p < .01$, $\eta^2 = .10$), and germane load ($F = 4.35$, $p < .05$, $\eta^2 = .04$. However, regarding WOE scaffolding, there was no significant difference between the groups for any of the three types of cognitive load. Interaction between both variables was significant on germane load ($F = 5.35$, $p < .05$; $\eta 2 = .04$), although there was no significant difference between intrinsic and extraneous loads (see Table 3).

Self-Regulation Skills

Students' self-regulation skills were assessed using a questionnaire at the beginning and end of course instruction. Table 2 shows means and SD on self-regulation skills and comparison groups at pre-test and post-tests. To examine the effects of the provided scaffolding on students' self-regulation skills over time, repeated two-way ANCOVA analyses were conducted with prior knowledge with covariate. The results indicated significant interaction effects of time and groups about self-regulation skills ($F = 3.17$, $p < .05$, $\eta^2 = .07$). Specifically, there are significant differences within groups in TWM

(F = 3.91, p < .05, η^2 = .03) and FWM (F = 6.84, p < .01, η^2 = .05). There are no significant differences within groups in TW (F = 1.20, p = .28, η^2 = .01) and FW (F = .23, p = .63, η^2 = .00).

Discussion

The Effects of Two Types of Scaffolding on Knowledge Acquisition

The TWM and FWM groups were found to be more effective than the TW and FW groups for knowledge acquisition. It was designed to focus on promoting metacognitive skills and cognitive skills together by using metacognitive scaffolding and WOE. Novices not only experience difficulties developing metacognitive skills themselves (Loksa et al., 2020), but they also need sufficient prior knowledge to effectively use metacognition. Thus, scaffolding that promotes learners' cognition and metacognitive together should be provided, despite the fact that most previous research focuses on promoting metacognitive skills and cognitive skills separately.

Meanwhile, in this study, TWM was more effective in acquiring knowledge than FWM. The FWM group had higher knowledge acquisition scores than the TW group, but the TW had a higher score than the FW group. Furthermore, traditional WOE showed a statistically significant main effect on knowledge acquisition. This result implies that providing faded WOE, which erases some parts of the learning content, does not directly promote the cognition of novices that leads to successful knowledge acquisition. Several previous studies have reported that faded WOE can impede learning in novices. It can be confirmed that the well-structured WOE with correct answers and step-by-step problem-solving guidance helped novice learners to accurately understand programming knowledge (Vieira et al., 2015). It can thus be confirmed that traditional WOE is useful for novice learners to acquire knowledge and that it can also be applied to the field of programming education.

The Effects of Two Types of Scaffolding on Problem-solving

The results indicated that the FWM group with metacognitive scaffolding and faded WOE together was the most effective for problem-solving. The result of this study showed that metacognitive scaffolding and faded WOE had a significant effect on problem-solving. These findings are consistent with previous research, which reported that faded WOE guides learners to effectively acquire important concepts and principles required at each phase based upon the expert's problem-solving process (Jung et al., 2019a; Renkl, 2002). These findings also support those of previous studies that metacognitive strategies and cognitive strategies should be used together for effective problem-solving (Loksa et al., 2016; Magana et al., 2019). Previous studies have shown that strategies that promote metacognitive skills, such as organizing and planning activities in the early phases of problem-solving processes, can help explore solutions in the next phase of problem-solving (Pedrosa et al., 2016; Schraw et al., 2006). The

results of this study suggest that providing metacognitive scaffolding suitable for each problem-solving phase can lead to successful knowledge transfer in novice learners.

Meanwhile, faded WOE has also led to successful knowledge transfer by helping learners promote cognitive skills required for problem-solving. Vieria and colleagues (2017) reported that WOE for programming learning is effective in acquiring the key concepts and main solution principles that help to solve new problems. In particular, a WOE, in which some important information was faded, had a positive effect on learning transfer (Chi et al., 1989; Renkl et al., 2004). Given that novices need sufficient prior knowledge to promote metacognitive effectively (Loksa et al., 2016; Loksa & Ko, 2016; Pedrosa et al., 2016), the findings suggest that the faded WOE promoted the cognitive aspect for problem-solving, leading to effective knowledge transfer.

This study found no interaction between faded WOE and metacognitive scaffolding. It seems to be because the cognitive scaffolding and metacognitive scaffolding did not design a close connection with each other. In other words, it can be inferred that the faded WOE content structure was not more closely linked with metacognitive strategies such as planning, monitoring, or reflection of each phase. Given that the interaction of cognitive skills and metacognitive skills is important for successful problem-solving (Magana et al., 2019), the metacognitive strategy and the faded WOE should be designed to work closely together based on the characteristics of each problem-solving phase.

The Effects of Cognitive Load

The findings indicated that metacognitive scaffolding significantly affected three types of cognitive load. Metacognitive scaffolding had the main effect, at a statistically significant level, on intrinsic and extraneous load and influenced the germane load at a small significance level. These results support previous research that metacognitive scaffolding contributes to reducing the cognitive load (Lopez-Vargas et al., 2017; Shin & Song, 2022). However, the WOE type had no significant effect on all three cognitive load types. These results support previous studies that metacognitive scaffolding contributes to reduced cognitive load. On the other hand, many previous studies provided faded WOE to reduce cognitive load but faded WOE alone did not affect cognitive load during problem-solving programming in this study. There was an interaction effect between faded WOE and the metacognitive scaffolding that significantly induced germane load. These results suggest that the FWM group, which scored the highest on the problem-solving skill, was able to appropriately adjust cognitive load with metacognitive scaffolding and faded WOE support.

Self-Regulation Skills

The findings indicated that the TWM and FWM groups were more effective in promoting self-regulation skills than the other groups. In particular, metacognitive scaffolding had a significant effect on self-regulation skills. These findings supported

previous research, which suggested that metacognitive scaffolding could help to promote self-regulation skills (Pedrosa et al., 2016; Schraw et al., 2006; Zimmerman & Schunk, 2011).

Experts solve problems effectively through planning, monitoring, and reflection processes, whereas novices lack such metacognitive skills (Loksa et al., 2020). Previous research indicated that novices need both cognitive and metacognitive scaffolding together because basic knowledge is required to effectively apply metacognitive strategy (Loksa et al., 2016; Loksa & Ko, 2016). The findings of this study imply that the WOE type that does not fade important information is effective for novices' self-regulation when WOE is provided with metacognitive scaffolding. On the other hand, considering the expertise reversal effect, which is a weakness of WOE (Kalyuga et al., 2003), it is necessary to refine appropriate fading WOE technique to enable novice learners to progress to skilled levels in programming education.

Conclusion and Limitations

This study has shown that providing metacognitive scaffolding with faded WOE (FWM) that can promote learner cognitive skills and metacognitive skills is the most effective for knowledge acquisition and problem-solving. In addition, it is also effective for extraneous, germane load, and self-regulation skills. These results provide empirical evidence that the combination of learning strategies to promote metacognitive and cognitive skills has a positive impact on the success of problem-solving programming learning. In addition, the faded WOE suitable for each problem-solving process can promote novices' cognition, despite the fact that previous studies reported that the faded WOE can lead to cognitive difficulties (Paas, 1992; van Merriënboer & Kirschner, 2012). In this study, we present empirical evidence that, in problem-solving programming learning, problem-solving skills can be improved by 1) cognitive scaffolding faded some parts of the content to support the acquisition of the concepts and principles needed for each phase of problem-solving and 2) metacognitive scaffolding to support planning for problem-analysis phase and to support monitoring and evaluating trial and error for the problem-solving phase.

There are limitations and future research implications associated with this study. First, it is necessary to generalize the research results by securing more research participants. Second, future studies need to propose to derive more sophisticated and correlated between faded WOE and metacognitive scaffolding design strategies based on the characteristics of each phase of problem-solving. Third, in this study, metacognitive scaffolding focused on planning, monitoring, and reflection. In future research, it is necessary to expand the metacognitive strategies such as motivation and self-explanation.

Declaration of conflicting interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) received no financial support for the research, authorship, and/or publication of this article.

ORCID iD

Yoonhee Shin https://orcid.org/0000-0001-6684-3404

References

Abdul-Rahman, S. S., & Du Boulay, B. (2014). Learning programming via worked-examples: Relation of learning styles to cognitive load. *Computers in Human Behavior*, *30*, 286–298. https://doi.org/10.1016/j.chb.2013.09.007

Agbo, F. J., Oyelere, S. S., Suhonen, J., & Adewumi, S. (2019, November). *A systematic review of computational thinking approach for programming education in higher education institutions*. Proceedings of the 19th koli calling international Conference on computing education research (pp. 1–10), NY, New York, Association for Computing Machinery. https://doi.org/10.1145/3364510.3364521

Chao, P. Y. (2016). Exploring students' computational practice, design and performance of problem-solving through a visual programming environment. *Computers and Education*, *95*, 202–215. https://doi.org/10.1016/j.compedu.2016.01.010

Chi, M. T., Bassok, M., Lewis, M. W., Reimann, P., & Glaser, R. (1989). Self-explanations: How students study and use examples in learning to solve problems. *Cognitive Science*, *13*(2), 145–182. https://doi.org/10.1207/s15516709cog1302_1

Crippen, K. J., & Earl, B. L. (2007). The impact of web-based worked examples and self-explanation on performance, problem-solving, and self-efficacy. *Computers and Education*, *49*(3), 809–821. https://doi.org/10.1016/j.compedu.2005.11.018

Garner, S. (2001). A tool to support the use of part-complete solutions in the learning of programming. In *Proceeding de conférence* (pp. 222–228).

Garner, S. (2002). Reducing the cognitive load on novice programmers. In P. Barker & S. Rebelsky (Eds.), *Proceedings of ED-MEDIA 2002–world conference on educational multimedia, hypermedia and telecommunications* (pp. 578–583). Association for the advancement of computing in education (AACE). Retrieved October 6, 2022 from https://www.learntechlib.org/primary/p/10329/

Ge, X., Law, V., & Tawfik, A. (2016). The design of scaffolding and fading: Research issues and challenges. In M. W. Minhong, P. Kirschner, & S. Bridges (Eds.), *The pre-conference workshop at the 12th international conference of the learning sciences* (pp. 19–24). The University of Hong Kong.

Haseski, H. İ., Ilic, U., & Tugtekin, U. (2018). Defining a new 21st century skill-computational thinking: Concepts and Trends.*International Education Studies*, *11*(4), 29–42. https://doi.org/10.5539/ies.v11n4p29, https://eric.ed.gov/?id=EJ1175216

Jung, J., Hong, J., & Shin, Y. (2019a). The effect of faded information based on learners' prior knowledge in problem-solving learning. *Korean Association For Learner-Centered Curriculum And Instruction*, *19*(15), 799–820. https://doi.org/10.22251/jlcci.2019.19.15.799

Jung, J., Jung, M. K., Kim, K. E., Kwon, A. R., Chae, H. W., Yoon, C. S., Kim, H. S., & Kim, D. H. (2016). Ultrasound measurement of pediatric visceral fat thickness: Correlations with metabolic and liver profiles. *Annals of Pediatric Endocrinology and Metabolism*, *21*(2), 75–80. https://doi.org/10.6065/apem.2016.21.2.75, https://www.jstor.org/stable/jeductechsoci.19.4.75

Jung, J., Shin, Y., & Zumbach, J. (2019b). The effects of pre-training types on cognitive load, collaborative knowledge construction and deep learning in a computer-supported. collaborative learning environment. *Interactive Learning Environments*, *29*(7), 1163–1175. https://doi.org/10.1080/10494820.2019.1619592

Kalyuga, S., Ayres, P., Chandler, P., & Sweller, J. (2003). The expertise reversal effect. *Educational Psychologist*, *38*(1), 23–31. https://doi.org/10.1207/s15326985ep3801_4

Leppink, J., Paas, F., Van der Vleuten, C. P., Van Gog, T., & Van Merriënboer, J. J. (2013). Development of an instrument for measuring different types of cognitive load. *Behavior Research Methods*, *45*, 1058–1072.

Lister, R., Simon, B., Thompson, E., Whalley, J. L., & Prasad, C. (2006). Not seeing the forest for the trees: Novice programmers and the SOLO taxonomy. In *Proceedings of the 11th annual sigcse conference on innovation and technology in computer science education* (pp. 118–122). ACM. https://doi.org/10.1145/1140124.1140157

Loksa, D., & Ko, A. J. (2016, August). *The role of metacognitive in programming problem. solving process and success*. Proceedings of the 2016 ACM conference on international computing education research (pp. 83–91), NY, New York, Association for Computing Machinery. https://doi.org/10.1145/1140124.1140157

Loksa, D., Ko, A. J., Jernigan, W., Oleson, A., Mendez, C. J., & Burnett, M. M. (2016, May). Programming, problem-solving, and self-awareness: Effects of explicit guidance. *Proceedings of the 2016 CHI Conference on human Factors in computing* (pp. 1449–1461). NY, New York, Association for Computing Machinery.

Loksa, D., Xie, B., Kwik, H., & Ko, A. J. (2020, February). *Investigating novices' in situ reflections on their programming process*. Proceedings of the 51st ACM Technical. Symposium on computer science education (pp. 149–155).

López-Vargas, O., Ibáñez-Ibáñez, J., & Racines-Prada, O. (2017). Students' metacognition and cognitive style and their effect on cognitive load and learning achievement. *Journal of Educational Technology and Society*, *20*(3), 145–157. https://www.jstor.org/stable/26196126

Magana, A. J., Fennell, H. W., Vieira, C., & Falk, M. L. (2019). Characterizing the interplay. of cognitive and metacognitive knowledge in computational modeling and simulation practices. *Journal of Engineering Education*, *108*(2), 276–303. https://doi.org/10.1002/jee.20264

Molenaar, I., van Boxtel, C. A., & Sleegers, P. J. (2011). Metacognitive scaffolding in an innovative learning arrangement. *Instructional Science*, *39*(6), 785–803. https://doi.org/10.1007/s11251-010-9154-1

Moreno, R. (2006). When worked examples don't work: Is cognitive load theory at an impasse? *Learning and Instruction*, *16*(2), 170–181. https://doi.org/10.1016/j.learninstruc.2006.02.006

National Academies of Sciences (2018). *How people learn II: Learners, contexts, and cultures*. National Academies Press.

Paas, F., Tuovinen, J. E., Tabbers, H., & Van Gerven, P. W. (2003). Cognitive load measurement as a means to advance cognitive load theory. *Educational Psychologist*, *38*(1), 63–71. https://doi.org/10.1207/s15326985ep3801_8

Paas, F. G. (1992). Training strategies for attaining transfer of problem-solving skill in. statistics: A cognitive-load approach. *Journal of Educational Psychology*, *84*(4), 429–434. https://doi.org/10.1037/0022-0663.84.4.429

Paas, F. G., Renkl, A., & Sweller, J. (2004). Cognitive load theory: Instructional implications of the interaction between information structures and cognitive architecture. *Instructional Science*, *32*(1/2), 1–8. https://doi.org/10.1023/b:truc.0000021806.17516.d0

Paas, F. G., & Van Merriënboer, J. J. (1994). Instructional control of cognitive load in the. training of complex cognitive tasks. *Educational Psychology Review*, *6*(4), 351–371. https://doi.org/10.1007/BF02213420

Pedrosa, D., Cravino, J., Morgado, L., & Barreira, C. (2016, June). Self-regulated learning in. Computer programming: Strategies students adopted during an assignment. In *International conference on immersive learning* (pp. 87–101). Springer.

Pintrich, P. R., Smith, D. A. F., Garcia, T., & McKeachie, W. J. (1991). *A manual for the use. Of the motivated Strategies for learning questionnaire (MSLQ) (tech. Report No. 91-B-004)*. Board of Regents, University of Michigan.

Prather, J., Becker, B. A., Craig, M., Denny, P., Loksa, D., & Margulieux, L. (2020, August). *What do we think we think we are doing? Metacognition and metacognitive in programming*. Proceedings of the 2020 ACM conference on international computing education research (pp. 2–13).

Puntambekar, S., & Hubscher, R. (2005). Tools for scaffolding students in a complex learning environment: What have we gained and what have we missed? *Educational Psychologist*, *40*(1), 1–12. https://doi.org/10.1207/s15326985ep4001_1

Renkl, A. (2002). Worked-out examples: Instructional explanations support learning by self-explanations. *Learning and Instruction*, *12*(5), 529–556. https://doi.org/10.1016/S0959-4752(01)00030-5

Renkl, A., Atkinson, R. K., & Große, C. S. (2004). How fading worked solution steps works–a cognitive load perspective. *Instructional Science*, *32*(1/2), 59–82. https://doi.org/10.1023/B:TRUC.0000021815.74806.f6

Renkl, A., Atkinson, R. K., & Maier, U. H. (2000, August). From studying examples to. solving problems: Fading worked-out solution steps helps learning. In L. Gleitman & A. K. Joshi (Eds), *In proceeding of the 22nd annual Conference of the cognitive science society* (pp. 393–398). Lawrence Erlbaum Associates, Inc.

Salleh, S. M., Shukur, Z., & Judi, H. M. (2018). Scaffolding model for efficient programming. learning based on cognitive load theory. *International Journal of Pure and Applied Mathematics*, *118*(7), 77–83.

Schraw, G., Crippen, K. J., & Hartley, K. (2006). Promoting self-regulation in science education: Metacognition as part of a broader perspective on learning. *Research in Science Education*, *36*(1-2), 111–139. https://doi.org/10.1007/s11165-005-3917-8

Shin, Y., & Song, D. (2022). The effects of self-regulated learning support on learners'. Task. Performance and cognitive load in computer programing. *Journal of Educational Computing Research*, *60*(6), 1490–1513. https://doi.org/10.1177/07356331211052632

Stachel, J., Marghitu, D., Brahim, T. B., Sims, R., Reynolds, L., & Czelusniak, V. (2013). Managing cognitive load in introductory programming courses: A cognitive aware scaffolding tool. *Journal of Integrated Design and Process Science*, *17*(1), 37–54. https://doi.org/10.3233/jid-2013-0004

Sweller, J., Van Merrienboer, J. J. G., & Paas, F. (1998). Cognitive architecture and. instructional design. *Educational Psychology Review*, *10*(3), 251–296. https://doi.org/10.1023/A:1022193728205

van Merriënboer, J. J., & Kirschner, P. (2012). *Ten steps to complex learning: A systematic. Approach to four-component instructional design* (2nd ed.). Routledge/Taylor and Francis Group.

van Rossum & Python development team (2018). *Python Tutorial Release 3.7.0*. Python Software Foundation.

Vieira, C., Yan, J., & Magana, A. (2015). Exploring design characteristics of worked examples to support programming and algorithm design. *The Journal of Computational Science Education*, *6*(1), 2–15. https://doi.org/10.22369/issn.2153-4136/6/1/1

Xie, B., Loksa, D., Nelson, G. L., Davidson, M. J., Dong, D., Kwik, H., Tan, A. H., Hwa, L., Li, M., Ko, A. J., & Ko, A. J. (2019). A theory of instruction for introductory programming skills. *Computer Science Education*, *29*(2-3), 205–253. https://doi.ort/10.1080/08993408.2019.1565235

Yousoof, M., Sapiyan, M., & Kamaluddin, K. (2007). Reducing cognitive load in learning. computer programming. *International Journal of Computer and Information Engineering*, *1*(12), 4100–4103.

Zimmerman, B., & Schunk, D. H. (2011). *Handbook of self-regulation of learning and performance*. Taylor and Francis.

Author Biographies

Yoonhee Shin is a professor in the Department of Educational Technology at Hanyang University. Professor Shin's research focus is on instructional design, learning analytics, computer-supported collaborative learning (CSCL), and software education including AI for everyone. Her work aims to enhance problem-solving skills, promote meaningful peer interaction, and foster computational thinking. Her current projects investigate the interplay of cognitive and meta-cognitive skills to develop optimized strategies for effective knowledge-building activities, with an emphasis on motivation in coding and higher education contexts.

Jaewon Jung is a research fellow of Office of Higher Education Research at Korean Educational Development Institute, South Korea. She holds a Ph.D. in Educational Technology from Hanyang University, South Korea. Her current research interests include exploring the educational issues such as multimedia learning, computer-supported collaborative learning (CSCL), problem-based learning (PBL), and

instructional design to provide effective learning strategies and enhance learning outcomes.

Joerg Zumbach received his Diploma in Psychology 1999 from Ruprecht-Karls-University Heidelberg, Germany. He got his Dr. phil. in Educational Psychology 2003 from Ruprecht-Karls-University Heidelberg, Germany. Since 2006 he is Full Professor for Science Teaching and Learning Research and e-Learning at the Paris-Lodron University Salzburg, Austria. He is in different editorial, authored and co-authored various research articles and text books in the areas of Multimedia and Hypermedia Learning, Higher Education, Problem-Based Learning, and others.

Eunseon Yi is a professor in the Department of Creative Convergence Education and in the Department of Computer Science at Hanyang University ERICA. Her research focuses on education data mining, AI for education, automated judgement of honest video watching, learning analytics, and AI technology in education for raise meta-cognitive.

View publication stats